



SHIP THE AGENT: ONE BINARY, ONE API, ONE PRODUCTION BOUNDARY

ШІ як сервіс: деплой ADK-агента єдиним бінарником.

#payments-support

go run . -mode=graph console

жодної адреси

один бінарник, одне API, одна межа

ЧОГО ВИ НАВЧИТЕСЬ СЬОГОДНІ

- 1 Пояснити, чому запущений у терміналі агент ще не є сервісом service boundary
- 2 Обгорнути власний workflow-агент із Дня 3 у HTTP-сервер через `adkrest.NewServer` same agent
- 3 Перевірити liveness, readiness і SSE-відповідь локально — `curl`, не слайд health · ready · sse
- 4 Коректно завершити сервер після `SIGTERM`: drain, обмежене очікування, чесний код виходу graceful
- 5 Зібрати статичний бінарник і **виміряти**, а не вгадати, розмір образу `FROM scratch` measure
- 6 🔥 Перевірити поведінку за паралельних SSE-запитів і знайти вузьке місце через `pprof` senior track

СПОЧАТКУ **200 OK**, ПОТІМ ТЕОРІЯ

```
$ cd ai-ae-labs/week2/Day4_Agent_as_Service_Deploy/labs4
$ go run .
# в іншому терміналі
$ curl -i http://localhost:8080/health
HTTP/1.1 200 OK
```

OK

Що доводить **200**

Процес доступний іншій програмі за **стабільною адресою**. Не якість відповіді моделі — доступність.

Чого тут НЕ має бути

Виклику LLM. Health-check дешевий, швидкий і не списує токени. Граф Дня 3 без LLM — тому й без ключа.

/readyz уже є: **200** у штатному режимі, **503** під час drain. Ключ не потрібен: сервіс використовує graph-шлях Дня 3.

СЕРВІС — ЦЕ ШІСТЬ ОБІЦЯНОК ІНШОМУ КОДУ

1 · Повторюваний запуск

Один артефакт, одна команда, той самий результат на іншій машині.

2 · Перевірка без моделі

`/health` не викликає LLM. Дешево, швидко, без токенів.

3 · Відомий контракт

Сесія, запит, події — за документованим форматом, не «якось».

4 · Поступова відповідь

SSE: клієнт бачить прогрес, сервер тримає кілька довгих з'єднань.

5 · Передбачувана зупинка

Не беру нову роботу, даю поточній завершитися, чесний код виходу.

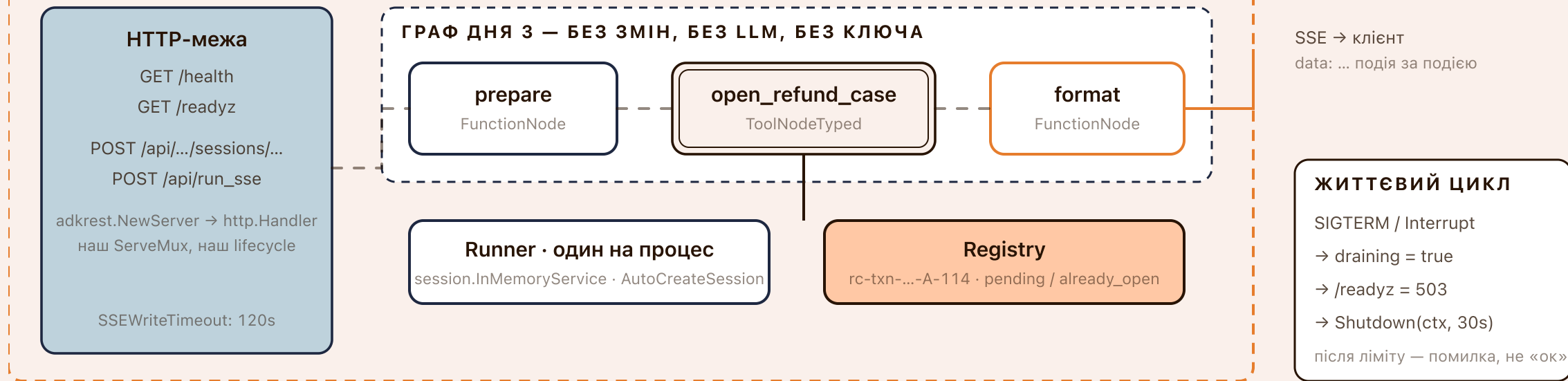
6 · Артефакт, який міряють

Статичний бінарник, мінімальний образ, виміряний розмір.

Ми не створюємо другого демо-агента. Беремо граф Дня 3 і додаємо йому **саме цю межу**. Артефакт курсу росте, а не розсипається.

ОДИН БІНАРНИК · ОДНЕ АРІ · ОДНА МЕЖА

FROM SCRATCH · ОДИН СТАТИЧНИЙ БІНАРНИК · РОЗМІР — ВИМІР, НЕ ОБІЦЯНКА



□ функція / runtime □ інструмент □ реєстр □ контейнер

ADKREST.NEWSEVER — ЦЕ ПРОСТО HTTP.HANDLER

```
restServer, err := adkrest.NewServer(adkrest.ServerConfig{
    AgentLoader:    agent.NewSingleLoader(a),      // а — ваш граф із Дня 3
    SessionService: session.InMemoryService(),
    SSEWriteTimeout: 120 * time.Second,           // окремий параметр для довгого стріму
})
if err != nil { log.Fatal(err) }

mux := http.NewServeMux()
mux.Handle("/api/", http.StripPrefix("/api", restServer)) // ADK під /api/, решта — наша
```

Звичайний `net/http`

Свій `ServeMux`, свої `/health` і `/readyz`, свій lifecycle. ADK не нав'язує HTTP-фреймворк.

`NewSingleLoader(a)`

Агент стає доступним за своїм ім'ям — і саме воно має збігатися з `appName` у кожному запиті (у шаблоні `first_graph_agent`).

ЖИВИЙ ≠ ГОТОВИЙ

```
// «процес живий?» — завжди дешево
mux.HandleFunc("/health",
    func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
        _, _ = w.Write([]byte("OK\n"))
    })

// «можна слати новий запит зараз?»
var draining atomic.Bool
mux.HandleFunc("/readyz",
    func(w http.ResponseWriter, r *http.Request) {
        if draining.Load() {
            http.Error(w, "not ready", http.StatusServiceUnavailable)
            return
        }
        _, _ = w.Write([]byte("ready\n"))
    })
```

/health · liveness

Процес живий. Не залежить від ключа, моделі, черги. Якщо почне віддавати 503 під час drain — зовнішня система вб'є процес раніше.

/readyz · readiness

Конфіг є, ключ є (якщо потрібен), сервер **не зупиняється**. Не робить це LLM-викликом — probe стане платним і нестабільним.

Контрольна точка junior-шляху: агент уже має адресу й перевірюваний стан. Решта робить адресу коректною під стрімом, зупинкою й навантаженням.

ЧОМУ НЕ ПРОСТО `POST /ask`

ПРИЗНАЧЕННЯ	МЕТОД І МАРШРУТ
Створити сесію	<code>POST /api/apps/{app_name}/users/{user_id}/sessions/{session_id}</code>
Запустити, отримати всі події після завершення	<code>POST /api/run</code>
Запустити, отримувати події як SSE	<code>POST /api/run_sse</code>
Liveness (наш)	<code>GET /health</code>
Readiness (наш)	<code>GET /readyz</code>

Тіло запиту — camelCase

`appName`, `userId`, `sessionId`, `newMessage` — так у фактичній `RunAgentRequest` ADK Go v2. Невідомі поля сервер **відхиляє**: контракт не вгадують.

Власний `/ask` — можна, але

тоді формат повідомлення, сесії, події та стрімінг — усе домовляти самому. `adkrest` це вже зробив; ми додаємо тонку оболонку.

ТОЙ САМИЙ ЗАПИТ, ЩО ВЧОРА — ТЕПЕР ІЗ МЕРЕЖІ

```
# 1. спершу сесія — інакше 404
$ curl -X POST http://localhost:8080/api/apps/first_graph_agent/users/demo/sessions/demo-1 -H 'Content-Type: application/json' -d '{}'
```



```
# 2. потім потік; -N — не буферизувати
$ curl -N -X POST http://localhost:8080/api/run_sse -H 'Content-Type: application/json' -d '{
  "appName": "first_graph_agent", "userId": "demo", "sessionId": "demo-1", "streaming": true,
  "newMessage": { "role": "user",
    "parts": [{"text": "Мерчанти А-114 просять повернення по транзакції txn-2026-07-118845"}] }
}'
```

Content-Type: text/event-stream
data: ... подія за подією, не один JSON наприкінці

Сесія спершу

ADK перевіряє її до запуску: немає — `404`. `appName` в URL і в тілі = ім'я агента.

Той самий текст

Учора — у `console`. Сьогодні — з мережі. Це й відкриває шлях боту в `#payments-support`.

Дві метрики

Час до **першої** події і повний час відповіді — різні числа, обидва корисні.

STREAMING ≠ ШВИДША МОДЕЛЬ. HTTP ≠ A2A

SSE змінює досвід, не вартість

Модель не думає швидше, інструменти не дешевшають. Клієнт бачить прогрес раніше; сервер тримає кілька довгих з'єднань.

`SSEWriteTimeout` — **окремий** параметр: загальним

`WriteTimeout` довгий стрім не налаштуєш.

Доступний по мережі ≠ A2A-агент

MCP — агент↔інструмент · A2A — агент↔віддалений агент · MCP Apps/AG-UI — межа з користувачем. Сьогодні — лише фундамент: процес, до якого **узагалі** можна звернутися.

120 секунд `SSEWriteTimeout` у шаблоні — стартова гіпотеза, а не універсальна константа. Поставте явну, **виміряну** межу.

SIGTERM ПОСЕРЕД СТРІМУ: ЧОТИРИ КРОКИ

```
srv := &http.Server{Addr: ":8080", Handler: mux}
stop, cancel := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM) // 1. зловити
defer cancel()
go func() { if err := srv.ListenAndServe(); err != nil && !errors.Is(err, http.ErrServerClosed) { log.Fatalf("serve: %v", err) } }()

<-stop.Done()
draining.Store(true) // 2. /readyz → 503, нову роботу не беремо
shutdownCtx, shutdownCancel := context.WithTimeout(context.Background(), 30*time.Second)
defer shutdownCancel()
if err := srv.Shutdown(shutdownCtx); err != nil { // 3. чекаємо активні — в межах ліміту
    closeErr := srv.Close()
    log.Fatal(errors.Join(fmt.Errorf("graceful shutdown: %w", err), closeErr)) // 4. після ліміту — помилка, не «ок»
}
```

Що гарантує **Shutdown**

Не приймає нові з'єднання, чекає активні в межах контексту. Не гарантує, що довільний виклик моделі впадеться в 30 с — це ваша межа компромісу.

Демонструвати, не проголошувати

Довгий SSE у терміналі 1 · **SIGTERM** у терміналі 2 · **/readyz** → 503 або listener закрито · чи завершився запит · код виходу.
Ctrl+C без активного запиту — не доказ.

«ДАВАЙТЕ ОДИН ENDPOINT» — І ЩО ЛАМАЄТЬСЯ

`/health` → 503 під час drain

Зовнішня система вирішує: процес аварійний — і **вбиває його раніше**, ніж він дозавершить активні запити. Ви втратили саме те, заради чого робили graceful shutdown.

`/readyz` завжди 200

Нові довгі запити **продовжують прилітати** у процес, який уже завершується. Вони обірвуться через 30 с разом із ним.

Живий — окремо. Готовий — окремо. Навіть у маленькій вправі: саме під час зупинки видно, чому це не формальність.

«ОБРАЗИ З ТЕГОМ **LATEST** У ПРОД НЕ ЇДУТЬ»

Вимога 1 · явний тег

latest означає: образ, який пройшов рев'ю, і образ, який запустився, — **не обов'язково той самий**. Під PCI DSS це нездавана позиція.

Вимога 2 · перелічуваний runtime

На питання «що саме всередині контейнера» має існувати відповідь, коротша за «ну, Ubuntu і ще щось».

Обидві вимоги ведуть в одне місце: **явний тег версії + мінімальний базовий образ**. Для Go це **FROM scratch** — один статичний бінарник, без інтерпретатора, без інструментарію збірки.

«Статичний» ≠ «без залежностей»: агенту, що йде по HTTPS до провайдера, потрібні CA-сертифікати і змінні середовища з ключами. Це й покаже наступний слайд.

ОДИН БІНАРНИК, NON-ROOT, ЯВНИЙ PORT

```
FROM --platform=$BUILDPLATFORM golang:1.27.1 AS build
ARG TARGETOS
ARG TARGETARCH
WORKDIR /src
COPY go.mod go.sum ./
COPY week2/internal/refund/ ./week2/internal/refund/
COPY week2/Day4_Agent_as_Service_Deploy/labs4/ ./week2/Day4_Agent_as_Service_Deploy/labs4/
RUN CGO_ENABLED=0 GOOS=$TARGETOS GOARCH=$TARGETARCH go build -mod=readonly -trimpath \
    -ldflags="-s -w" -o /out/adk-service ./week2/Day4_Agent_as_Service_Deploy/labs4

FROM scratch
COPY --from=build /out/adk-service /adk-service
COPY --from=build /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/ca-certificates.crt
USER 65532:65532
ENV PORT=8080
EXPOSE 8080
ENTRYPOINT ["/adk-service"]
```

Build context — корінь `ai-ae-labs`. Allowlist `Dockerfile.dockerignore` не пропускає секрети. Числовий UID не потребує запису в `/etc/passwd`.

РОЗМІР — РЕЗУЛЬТАТ ВИМІРУ, НЕ ЧИСЛО НА СЛАЙДІ

```
$ docker build --platform linux/amd64 \
  -f week2/Day4_Agent_as_Service_Deploy/labs4/Dockerfile \
  -t adk-agent-service:local . # з кореня ai-ae-labs
$ docker image ls adk-agent-service:local
$ docker run --rm -p 127.0.0.1:8080:8080 adk-agent-service:local
```

Марко отримав ≈ 15 MB

Це його вимір на його машині та його піні ADK — так звучить у сценарії. Не характеристика фреймворку і не ваше очікуване число.

Що записати в README

Фактичний розмір · платформа · Go-версія · дата виміру · команда. Розмір залежить від версії ADK, лінкера, архітектури, CA bundle і вашого коду.

Малий образ — не самоціль. Важливо, що склад runtime **прозорий і відтворюваний**. Тому в рубриці — «виміряний розмір», а не «до N MB».

LAUNCHER ≠ НАШ HTTP-КОНТРАКТ

РЕЖИМ	КЛЮЧ У LAUNCHER	ДЛЯ ЧОГО ЗГАДУЄМО
Консоль	<code>console</code>	локальна взаємодія — День 3
REST API	<code>web api</code>	готовий API launcher (у нас — власна оболонка над <code>adkrest</code>)
A2A	<code>web a2a</code>	віддалений агент — Вебінар 10
Web UI	<code>web webui</code>	інтерфейс, не мета лаби
Agent Engine	<code>web agentengine</code>	керований рівень — лише pointer

Ключа `restapi` не існує — у коді це `api`. Секрет іде `env` → локальний `.env` у `.gitignore` → `secret manager`, але не в `YAML`, `Dockerfile` чи образ. Станом на 09/2026, v2.4.0.

Драбина деплою 2026 (станом на 08/2026): керована платформа → serverless-контейнер → Kubernetes → Docker-хост (наш рівень). Назви протухають швидше за код: Google — «Agent Engine → Agent Runtime → managed runtime у Gemini Enterprise» [secondary]; Microsoft Foundry, не Azure AI Foundry; одиниця деплою агента ≠ Pod: AWS microVM 8 год vs instances 14 діб, Azure скейлить на сесію. kagent v0.10.0 у RC + breaking changes (10.08) — лише forward pointer, без названих версій; agentgateway — стабільна лінія v1.4.1 (29.07), v1.5.0-beta.1 (25.08) — лише forward pointer, без названих версій.

ВИМІРЯТИ ПАРАЛЕЛЬНІСТЬ — І ЗНАЙТИ, ДЕ ГАРЯЧЕ

Навантаження: два чесні режими

stub-модель — міряємо обгортку й SSE (для нашого графа LLM взагалі немає); або **реальний провайдер** з малим бюджетом, підписаний як end-to-end. Для 1 / 5 / 10: завершені, TTFB/перша подія, повна тривалість, помилки, режим.

`pprof` — лише на `localhost`

Окремий debug-сервер на `127.0.0.1:6060`; blank import до нашого `ServeMux` сам не прикріпиться. `go tool pprof .../profile?seconds=20` під контрольованим навантаженням. Ніколи публічно.

У `labs4/service_test.go` — реальні HTTP/SSE, StateDelta й drain-тести. Навантаження та `pprof` — окреме розширення; не підмінюйте вимір зеленим unit-тестом.

Поганий висновок: «Go повільний». Добрий: «за 10 паралельних stub-SSE профіль показав X у JSON/логуванні; після зміни Y вимір Z став таким». Профіль — «де CPU»; trace (Тиждень 4) — «який крок агента».

ЩО В НАС ТЕПЕР Є — І ЧОГО ЩЕ НЕМАЄ

Є

Один артефакт, який запускається · сесія і запит · SSE-потік подій · liveness / readiness · коректна зупинка · перевірено під навантаженням і спрофільовано. Бот у `#payments-support` має до чого підключатися.

Ще немає — і сьогодні не вдаємо

Кластерної платформи · A2A-політики · identity · gateway · durable execution. kagent (CNCF Sandbox) запускатиме, agentgateway (Agentic AI Foundation, aaif.io) керуватиме трафіком, identity/policy визначатимуть права — у наступних курсах.

1

`adkrest.NewServer` — `http.Handler`; API під `/api/`, SSE після створення сесії

contract

2

Liveness $\neq$ readiness $\neq$ shutdown: зупинка = drain нових + обмежене очікування активних

lifecycle

3

`FROM scratch` з явним тегом: CA bundle і non-root — видимі залежності; розмір — вимір

artifact

АГЕНТ ВИХОДИТЬ У МЕРЕЖУ: HTTP-СЕРВІС І БІНАРНИК FROM SCRATCH

1 Обгорнути **власного** агента з ДЗ 3 через `adkrest`:
`/api/`, `/health`, `/readyz` 30

2 SSE через `run_sse` після сесії · graceful shutdown із демо `SIGTERM` у README 20

3 `FROM scratch` з явним тегом · статичний бінарник · **вимірний** розмір · CA та non-root явно 20

4 README: 4 `curl`-перевірки (liveness, readiness, сесія, SSE) + інструкція збірки 10

Базове = 80 балів

30 + 20 + 20 + 10. Сертифікат — від 60% за весь курс.
Бонус для сертифіката не потрібен.

++ Advanced — +20 додаткових балів

Навантаження SSE (k6/vegeta, таблиця) · `pprof` на loopback · scratch vs distroless vs alpine — виміряно.

Здача: GitHub-репо з кодом, Dockerfile, README; `go build ./...` проходить; посилання — у форму. Дедлайн — два тижні від відкриття завдання (рекомендуємо — протягом тижня).

АГЕНТ МАЄ АДРЕСУ. АЛЕ ЗНАЄ ЛИШЕ ТЕ, ЩО ЗНАЄ МОДЕЛЬ.

У LEDGERWORKS — 5 000 PDF: звіти PCI DSS, vendor SOC2, договори з еквайрами.

Оксана питає: «а таблицю ставок комісій із договору він прочитає?». Спойлер:

`pdftotext` знищує таблиці. Чанки по 800 токенів — розривають їх між векторами.

Сьогодні

Адреса · контракт · lifecycle · артефакт,
який міряють

Тиждень 3 · День 5

PDF → граф знань: lossless parsing,
ontology-driven chunking, entity resolution

Чому це боляче

Порізані таблиці → LLM здогадується
замість відповідати. Побачимо наживо.

ПИТАННЯ?

Форум курсу відкритий. Три підказки, з чого почати: чому ваш `/readyz` віддає `503`; що сталося на `SIGTERM` посеред стріму; яке число розміру образу вийшло у вас — і від чого воно залежить.

`/health` ≠ `/readyz`

`SIGTERM` → `drain` → `Shutdown`

`FROM scratch` + явний `тег`

розмір — вимір